SYNTH-02: Browser Monitors

> **Series:** SYNTH — Synthetic Monitoring | **Notebook:** 2 of 6 | **Created:** December 2025 | **Last Updated:** 07/01/2026

Creating and Optimizing Browser-Based Synthetic Tests

This notebook covers browser monitors in Dynatrace, including single-URL monitors, browser clickpaths, and performance analysis using the latest Dynatrace platform capabilities.

Table of Contents

Prerequisites

- ✅ Access to a Dynatrace environment with Synthetic Monitoring
- ✅ Completed SYNTH-01 Fundamentals
- ✅ Web application URL to monitor

> **How browser data is queried:** Browser monitors are analyzed primarily through the **`dt.synthetic.browser.*` metrics** (`timeseries`) — availability, total duration, and per-step duration. Execution-level browser records appear as `browser_monitor_execution` / `browser_step_execution` events in `fetch dt.synthetic.events` **when the classic browser experience is in use**; with the *new browser monitor experience* activated, detailed actions surface in RUM instead. **Dynatrace environments created after January 26, 2026 do not have a classic/new toggle at all** — they run the new experience by default, so this discovery step matters mainly for pre-2026-01-26 tenants. Confirm which path your tenant populates with a discovery query (`fetch dt.synthetic.events, from:-24h | filter startsWith(event.type, "browser") | limit 5`).

1. Browser Monitor Types

Single-URL Browser Monitor

Loads a single page and captures performance metrics:

Feature	Description
Execution	Full Chrome browser render
Metrics	W3C Navigation Timing, resource timing
Screenshots	Automatic capture on completion/failure
Validation	Content validation, element checks

Best For:
- Homepage availability
- Landing page performance
- Single page applications (SPA) initial load

Browser Clickpath Monitor

Multi-step user journey simulation:

Feature	Description
Steps	Multiple pages/actions in sequence
Interactions	Click, type, select, wait
State	Cookies maintained within execution (session NOT maintained between executions)
Content Validation	Validate text/elements exist on page

> **Note:** Unlike RUM session properties, browser clickpaths do not support extracting data into variables for use across executions. Content validation allows you to verify expected text/elements exist, but you cannot capture dynamic values for reuse.

Best For:
- Login flows
- Checkout processes
- Form submissions
- Multi-page workflows

Configuration Path

Dynatrace menu → Synthetic → Create synthetic monitor → Create browser monitor

2. Single-URL Monitors

Creating a Single-URL Monitor

1. **URL Configuration**
   - Enter the full URL (https://...)
   - Set viewport size (desktop, tablet, mobile)
   - Configure user agent string
2. **Execution Settings**
   - Frequency: 5-60 minutes

- Locations: Select public or private
- Timeout: Maximum execution time

3. **Validation Rules**
   - HTTP status code validation
   - Content validation (text, regex)
   - Element presence checks

Viewport Presets

Preset	Dimensions	Use Case
Desktop	1920x1080	Standard desktop
Laptop	1366x768	Common laptop
Tablet	768x1024	iPad portrait
Mobile	375x667	iPhone 8

```
// List browser/clickpath monitors (these are synthetic_test entities)
fetch dt.entity.synthetic_test
| fields id, entity.name
| sort entity.name asc
| limit 50
```

```
// Browser monitor availability + duration (last 24h) — metrics path
// Metric duration is already in milliseconds
timeseries {
    availability_pct = avg(dt.synthetic.browser.availability),
    duration_ms      = avg(dt.synthetic.browser.duration)
  }, from: now() - 24h, interval: 1h, by: {dt.entity.synthetic_test}
```

3. Browser Clickpaths

Creating Clickpath Monitors

Option 1: Record with Browser Extension

1. Install Dynatrace Synthetic Recorder (Chrome extension)

2. Start recording session
3. Perform user journey in browser
4. Stop recording and export to Dynatrace

Option 2: Manual Script Creation

Define steps programmatically using the script editor.

Common Actions

Action	Description	Example
`navigate`	Go to URL	Navigate to login page
`click`	Click element	Click login button
`type`	Enter text	Type username
`selectOption`	Select dropdown	Select country
`wait`	Wait for condition	Wait for element visible
`javascript`	Execute JS	Custom validation

Element Selectors

Selector Type	Example	Best Practice
CSS	`#login-btn`	Preferred - stable
XPath	`//button[@id='login']`	Complex structures
Link Text	`Login`	Simple links
Data Attribute	`[data-testid='login']`	Test automation

```
// Clickpath step performance — per-step duration (metrics path)
timeseries step_duration_ms = avg(dt.synthetic.browser.step.duration),
    from: now() - 24h, interval: 1h, by: {dt.entity.synthetic_test,
step.name}
```

4. Performance Metrics

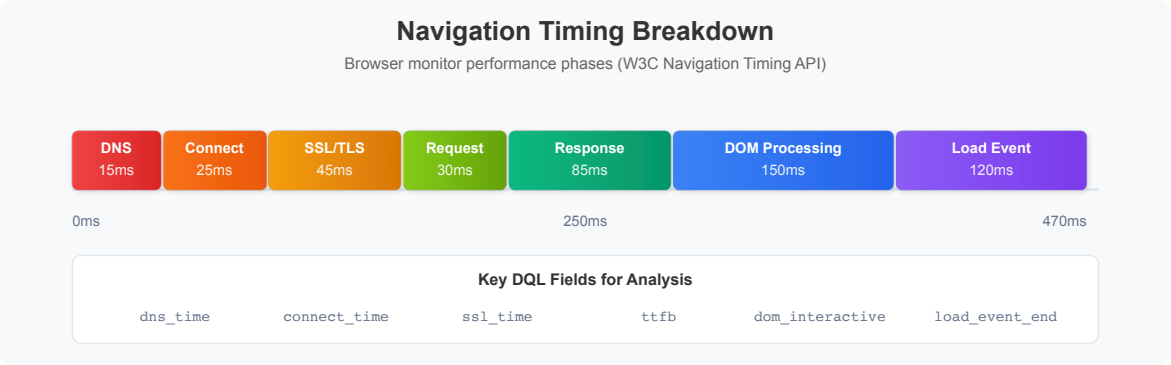
Browser timing in Synthetic on Grail

Browser monitors capture full-page render timing. In Grail this is exposed as **synthetic browser metrics**:

Metric key	Description	Unit
`dt.synthetic.browser.duration`	Total monitor duration (sum of all step durations). Dynatrace recommends this as the best representation of user experience.	ms
`dt.synthetic.browser.step.duration`	Individual step (action) duration	ms
`dt.synthetic.browser.availability`	Availability rate	%
`dt.synthetic.browser.executions` / `.step.executions`	Execution counts	count
`dt.synthetic.browser.classic.total_duration` / `.step.classic.total_duration`	Total duration measured by the **classic RUM JavaScript** — kept available even after the new experience is activated (dual ingestion, no extra cost)	ms
`dt.synthetic.browser.user_events.duration` / `.step.user_events.duration`	Duration measured from the new RUM JavaScript, summed from step-level user events	ms

Metric key	Description	Unit
`dt.synthetic.browser.user_events.total_duration` / `.step.user_events.total_duration`	Total duration from the new RUM JavaScript — the extended/enhanced breakdown that ships with the new browser monitor experience	ms

> **New-experience metric family:** the `user_events.*` and `classic.total_duration` metrics only appear once a monitor is executing under the **new browser monitor experience** (environment- or monitor-level setting, or by default on tenants created after January 26, 2026). Both classic and new-experience metrics are ingested simultaneously during the transition, at no additional cost — so it's safe to query for `user_events.*` speculatively and fall back to plain `duration` if it returns nothing.



Core Web Vitals — a note on where they live

Metric	Description	Good	Needs Work
First Contentful Paint	First content rendered	< 1.8s	> 3.0s
Largest Contentful Paint	Largest element rendered	< 2.5s	> 4.0s
Cumulative Layout Shift	Visual stability	< 0.1	> 0.25

> **Important:** Core Web Vitals (LCP, FCP, CLS) are **Real User Monitoring** measurements, not synthetic browser metric keys. The Synthetic on Grail browser metrics expose **availability**, **total duration** (classic and new-experience variants), and **per-step duration** — not the individual W3C navigation-timing phases or Web Vitals. For Web Vitals analysis, see the WEBRUM series. Targets above are Google's thresholds; set your own baselines from observed data.
>
> **Sources:** Browser monitor metrics in Synthetic on Grail (DT docs), Activate new browser monitor experience (DT docs). Metric table (incl. `user_events.*` / `classic.total_duration` family) and dual-ingestion/no-extra-cost claim confirmed at source 07/01/2026.

```
// Browser duration — average and worst-case per monitor (metrics path)
timeseries {
    avg_duration_ms = avg(dt.synthetic.browser.duration),
    max_duration_ms = max(dt.synthetic.browser.duration)
  }, from: now() - 24h, interval: 1h, by: {dt.entity.synthetic_test}
```

```
// Performance trend over time (last 7 days) — metrics path
timeseries {
    avg_duration_ms = avg(dt.synthetic.browser.duration),
    max_duration_ms = max(dt.synthetic.browser.duration)
  }, from: now() - 7d, interval: 1h
```

5. Validation and Assertions

Content Validation

Validation Type	Description	Example
Text Present	Page contains text	"Welcome"
Text Absent	Page doesn't contain	"Error"
Regex Match	Pattern matching	`Order #\d{6}`
Element Exists	DOM element present	`#success-message`
Element Content	Element has text	Button says "Submit"

HTTP Validation

Check	Default Behavior	Customization
Status Code	Fails on 4xx/5xx (400-599)	Can configure to ignore specific codes
Response Body	Content validation	Text/regex matching
Screenshots	Captured on success/failure	Automatic

> **Note:** Response size validation and header validation are not available as built-in options. Use content validation or JavaScript steps for advanced checks.

Visual Validation

- Automatic screenshots on success/failure
- Visual comparison (pixel diff)
- Layout validation

```
// Failed browser executions with detail — events path
// Requires classic browser execution events (browser_monitor_execution) in
dt.synthetic.events.
// If your tenant uses the new browser experience, this returns no rows —
analyze failures via
// the dt.synthetic.browser.availability metric (dips below 100) instead.
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "browser_monitor_execution"
| filter result.state == "FAIL"
| fields timestamp,
         monitor = monitor.name,
         location = entityName(dt.entity.synthetic_location),
         status = result.status.message,
         detail = result.status.details
| sort timestamp desc
| limit 50
```

6. Analyzing Browser Results

```
// Browser availability by location (metrics path)
timeseries availability_pct = avg(dt.synthetic.browser.availability),
    from: now() - 24h, interval: 1h, by: {dt.entity.synthetic_test,
dt.entity.synthetic_location}
```

```
// Browser duration distribution by location (metrics path)
timeseries {
    avg_ms = avg(dt.synthetic.browser.duration),
    max_ms = max(dt.synthetic.browser.duration)
  }, from: now() - 24h, interval: 1h, by: {dt.entity.synthetic_location}
```

```
// Slowest browser executions (outliers) — events path
// Classic browser execution events only; see the note on cell above.
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "browser_monitor_execution"
| filter result.state == "SUCCESS"
| fieldsAdd duration_ms = result.statistics.duration / 1ms
| filter duration_ms &gt; 5000  // &gt; 5 seconds
| fields timestamp,
         monitor = monitor.name,
         location = entityName(dt.entity.synthetic_location),
         duration_ms
| sort duration_ms desc
| limit 20
```

Summary

In this notebook, you learned:

✅ **Browser monitor types** - Single-URL vs clickpath monitors
✅ **Creating monitors** - URL configuration, viewports, locations
✅ **Clickpath automation** - Recording, actions, selectors
✅ **Performance metrics** - `dt.synthetic.browser.*` metric keys (availability, duration, step duration)
✅ **Where Web Vitals live** - RUM, not synthetic browser metrics
✅ **Analysis queries** - Availability, duration trends, and failure diagnostics

Next Steps

Continue to **SYNTH-03: HTTP Monitors** to learn about lightweight API monitoring.

References

- Browser monitors (DT docs)
- Activate new browser monitor experience (DT docs)
- Browser monitor metrics in Synthetic on Grail (DT docs)
- Synthetic app (DT docs)